

Capítulo 3

Núcleos de Procesamiento. Paralelismo a Nivel de Instrucción

3.1 Introducción: del pipeline a la emisión múltiple

Conexión con temas anteriores Temas 1 y 2

En el Tema 1 se estableció que la métrica de rendimiento fundamental es el tiempo de ejecución, y que:

$$T_{\text{CPU}} = N \cdot \text{CPI} \cdot T_{\text{ciclo}}$$

En el Tema 2 se estudió la segmentación de cauce, que permite acercar el CPI al valor ideal de 1 explotando el **paralelismo entre etapas**. El Tema 3 lleva esta idea al siguiente nivel: si en lugar de una sola instrucción por ciclo se pueden emitir *varias*, el CPI puede ser menor que 1. En ese caso, la métrica de rapidez habitual se invierte y se usa el **IPC** (instrucciones por ciclo) en lugar del CPI.

El **Paralelismo a Nivel de Instrucción** (ILP, *Instruction-Level Parallelism*) es la capacidad de ejecutar varias instrucciones de forma simultánea dentro de un único núcleo de procesamiento. La segmentación que estudiamos en el Tema 2 es la forma más básica de ILP: en un pipeline de 5 etapas en estado estacionario, hay 5 instrucciones en vuelo simultáneamente, aunque cada una esté en una etapa distinta.

El CPI efectivo de un pipeline con todas sus fuentes de parada es:

$$\text{CPI}_{\text{ef}} = \text{CPI}_{\text{ideal}} + \underbrace{\text{paradas estructurales}}_{\Delta\text{CPI}_{\text{est}}} + \underbrace{\text{paradas por datos}}_{\Delta\text{CPI}_{\text{datos}}} + \underbrace{\text{paradas por control}}_{\Delta\text{CPI}_{\text{ctrl}}}$$

El objetivo del ILP es reducir al máximo cada uno de estos términos. Las técnicas del Tema 2 (forwarding, predicción de saltos, delayed branch) atacaban los ΔCPI manteniendo el $\text{CPI}_{\text{ideal}} = 1$. Las técnicas del Tema 3 van más allá: reducen el $\text{CPI}_{\text{ideal}}$ por debajo de 1 mediante la **emisión múltiple**.

Definición Emisión múltiple

La **emisión múltiple** (*multiple issue*) es la capacidad del procesador de iniciar la ejecución de más de una instrucción en el mismo ciclo de reloj. Requiere replicar partes del hardware del pipeline para crear múltiples caminos de datos paralelos. Con emisión de n instrucciones por ciclo, el IPC máximo teórico es n (CPI mínimo = $1/n$).

La emisión múltiple se puede implementar de dos formas fundamentalmente distintas, con implicaciones profundas sobre quién (compilador vs. hardware) se encarga de encontrar y gestionar el paralelismo:

Cuadro 3.1: Comparativa entre emisión múltiple estática y dinámica

	Estática	Dinámica (superescalar)
Responsable de agrupar	Compilador	Hardware (CPU)
Detección de riesgos	Compilador (en tiempo de compilación)	Hardware (en tiempo de ejecución)
Reordenamiento	Compilador	CPU (fuera de orden)
Complejidad hardware	Baja	Alta
Consumo de potencia	Bajo	Alto
Adaptación a fallos caché	No (no predecibles en compilación)	Sí (gestión dinámica)
Ejemplos comerciales	VLIW, IA-64 (Itanium)	Intel Core i7, ARM Cortex-A8
Uso típico	Aplicaciones científicas	Servidores, escritorio, móvil

3.2 Dependencias de nombre: revisión necesaria para el ILP

Antes de entrar en las técnicas de emisión múltiple, es imprescindible comprender con precisión las **dependencias de nombre**, pues son las que el renombrado de registros (técnica central de este tema) se encarga de eliminar.

Definición Dependencia de nombre

Dos instrucciones tienen una **dependencia de nombre** cuando usan el mismo registro (o dirección de memoria) pero entre ellas no hay flujo real de información: el registro es simplemente un *nombre* que comparten por accidente de compilación. No es una dependencia de datos verdadera, pero fuerza a

preservar el orden de ejecución, limitando el ILP explotable.

Las dependencias de nombre ya fueron presentadas en el Tema 2, pero allí no suponían riesgos en el MIPS de 5 etapas (las escrituras siempre llegan a WB después de las lecturas en ID). En procesadores con emisión dinámica y ejecución fuera de orden, las dependencias de nombre sí pueden generar riesgos WAR y WAW reales. Veamos los tres tipos y sus consecuencias:

3.2.1 RAW: dependencia verdadera (no es de nombre)

Aunque RAW (*Read After Write*) no es una dependencia de nombre (hay flujo real de información), se incluye aquí para comparación. El valor producido por la instrucción *i* es realmente necesario para la instrucción *j*. No se puede eliminar con renombrado; el único tratamiento es esperar a que el dato esté disponible o usar forwarding.

```
1 add x1, x2, x3    # escribe x1
2 sub x4, x1, x5    # lee x1: dependencia RAW en x1
```

Listing 3.1: RAW: dependencia verdadera, no eliminable por renombrado

3.2.2 WAR: antidependencia (dependencia de nombre)

En una **antidependencia** WAR (*Write After Read*), la instrucción *j* quiere escribir en un registro que la instrucción *i* aún no ha leído. Si se ejecutaran en el orden original no habría problema, pero si se permite reordenar para explotar ILP, *j* podría sobrescribir el registro antes de que *i* lo lea, obteniendo un resultado incorrecto. Es una dependencia *de nombre* porque si se renombrara el registro destino de *j*, la instrucción *i* seguiría leyendo el valor correcto y *j* podría escribir sin conflicto.

```
1 fdiv.d f0, f2, f4    # lee f0 (en realidad, lo escribe)
2 fadd.d f6, f0, f8    # lee f0: instruccion i
3 fsub.d f8, f10, f14   # escribe f8: instruccion j
4 fmul.d f6, f10, f8    # lee f8
5 # WAR en f8: fadd.d debe leer f8 antes de que fsub.d lo
  # sobrescriba
6 # WAR en f6: fmul.d escribiria f6, pero fadd.d ya lo leyo
```

Listing 3.2: WAR: antidependencia, eliminable por renombrado

3.2.3 WAW: dependencia de salida (dependencia de nombre)

En una **dependencia de salida** WAW (*Write After Write*), dos instrucciones escriben en el mismo registro. Si se ejecutan fuera de orden, la más reciente podría escribir antes que la más antigua, y el valor final en el registro sería el de la más antigua (incorrecto). Con renombrado, cada instrucción escribe en un registro diferente, y el registro arquitectónico visible al programador recibe el valor correcto al final.

```
1 add x1, x2, x3    # escribe x1 (primera escritura)
2 beq x4, x0, L
3 sub x1, x1, x6    # escribe x1 (segunda escritura): WAW en x1
```

4 L: or x7, x1, x8 # lee x1: debe ser el valor de sub, no de add

Listing 3.3: WAW: dependencia de salida, eliminable por renombrado

Concepto clave El renombrado de registros elimina WAR y WAW

Las dependencias WAR y WAW son **artificiales**: son consecuencia de la reutilización de nombres de registros por el compilador, no de flujos de datos reales. La técnica de **renombrado de registros** elimina ambas asignando un registro físico diferente a cada escritura, liberando al hardware para reordenar y ejecutar las instrucciones sin riesgo de corrupciones.

3.3 Emisión múltiple estática

3.3.1 Concepto y planificación

En la emisión múltiple estática, el **compilador** es responsable de examinar el código, identificar instrucciones independientes entre sí, y agruparlas en **paquetes de emisión** (*issue packets*). Cada paquete contiene exactamente las instrucciones que el hardware puede ejecutar en paralelo en un ciclo de reloj, determinadas por los recursos del pipeline disponibles (una ALU, una unidad de carga/almacenamiento, etc.).

Las reglas que debe respetar el compilador al construir los paquetes son:

1. Dentro de un mismo paquete, las instrucciones deben ser **completamente independientes** entre sí (no puede haber ninguna dependencia RAW, WAR ni WAW entre ellas), pues se ejecutan simultáneamente.
2. Entre paquetes consecutivos puede haber dependencias, pero el compilador debe conocer las latencias específicas del hardware para intercalar instrucciones de forma que esas dependencias no generen paradas.
3. Si no se encuentran instrucciones independientes suficientes para llenar el paquete, se insertan instrucciones **nop** (no-operación).

3.3.2 MIPS con emisión dual estática

El caso más sencillo y pedagógico de emisión estática es el **MIPS con emisión dual**: se emiten dos instrucciones por ciclo, agrupadas en un paquete de 64 bits (2×32 bits) con la siguiente restricción estructural fija:

- **Primera ranura (slot 0):** debe contener una instrucción **ALU** o de salto (tipo R, tipo I aritmético, **beq**, **bne**, **j**).
- **Segunda ranura (slot 1):** debe contener una instrucción **de acceso a memoria** (**lw**, **sw**, **ldc1**, **sdc1**).
- Si alguna ranura no puede llenarse con una instrucción útil, se inserta un **nop** en esa posición.

[INSERTAR FIGURA]

Diagrama temporal de emision dual estatica. Ciclo 1: [ALU-paq1/IF] y [LW-paq1/IF] simultaneamente. Ciclo 2: ambas en ID. Ciclo 3: ambas en EX. Ciclo 4: ambas en MEM. Ciclo 5: ambas en WB. En paralelo, paquete 2 entra en IF en ciclo 2. El IPC maximo es 2.

Figura 3.1: Diagrama temporal del MIPS con emision dual estatica. Cada par de instrucciones (ALU + Load/Store) avanza en paralelo por las 5 etapas.

[INSERTAR FIGURA]

Diagrama del camino de datos del MIPS con emision dual estatica. Se muestran dos caminos paralelos: uno para instrucciones ALU (multiplexores, ALU, banco de registros) y otro para instrucciones Load/Store (unidad de carga, caché de datos). Ambos comparten el banco de registros pero tienen acceso independiente. Fuente: diapositivas Tema 3.

Figura 3.2: Camino de datos del MIPS con emision dual estatica. Las etapas IF e ID se replican para poder buscar y decodificar dos instrucciones por ciclo.

3.3.3 Riesgos adicionales en la emision dual

La emision dual introduce nuevos casos de riesgo que no existían en la emision simple:

1. **Riesgo entre instrucciones del mismo paquete:** si una instrucción ALU produce un resultado que la instrucción Load/Store del mismo paquete necesita como dirección base, se produce un riesgo RAW intra-paquete. El compilador debe detectarlo y dividir el par en dos paquetes distintos.

```
1  # INCORRECTO: mismo paquete, riesgo RAW en $t0
2  [add $t0, $s0, $s1]  [load $s2, 0($t0)]  # $t0 no esta listo
3
4  # CORRECTO: paquetes separados
5  [add $t0, $s0, $s1]  [nop]                # ciclo 1
6  [nop]                [load $s2, 0($t0)]  # ciclo 2
```

Listing 3.4: Riesgo intra-paquete: debe dividirse en dos ciclos

2. **Riesgo load-use entre paquetes:** igual que en la emision simple, una instrucción de carga no puede proporcionar su resultado a la instrucción inmediatamente siguiente mediante forwarding. Pero ahora, como se emiten dos instrucciones por ciclo, la “instrucción inmediatamente siguiente” puede ser la instrucción ALU del siguiente paquete o la instrucción de carga del siguiente paquete. En ambos casos se necesita 1 ciclo de parada.

¡Atención! La planificación para emisión dual es más exigente que para emisión simple

En emisión simple, el compilador solo necesita separar instrucciones dependientes con instrucciones independientes intercaladas. En emisión dual, además debe asegurarse de que las dos instrucciones *dentro del mismo paquete* sean completamente independientes entre sí y con respecto a instrucciones de paquetes anteriores que puedan haber generado resultados aún no disponibles.

3.3.4 Ejemplo de planificación para MIPS dual

Ejemplo Planificación de bucle para MIPS dual estático

Código original:

```

1 Loop: lw    $t0, 0($s1)      # carga elemento array
2      addu   $t0, $t0, $s2    # suma escalar
3      sw     $t0, 0($s1)      # almacena resultado
4      addi   $s1, $s1, -4     # decrementa puntero
5      bne    $s1, $zero, Loop # salto si no ha terminado

```

Planificación para MIPS dual (ALU+BRANCH / LOAD+STORE), con 1 ranura de salto retardado:

Cuadro 3.2: Bucle planificado para MIPS dual estático ($IPC = 5/4 = 1,25$)

Ciclo	Slot ALU/Salto	Slot Load/Store	Notas
1	nop	lw \$t0, 0(\$s1)	carga
2	addi \$s1, \$s1, -4	nop	cubre latencia lw; actualiza ptr
3	addu \$t0, \$t0, \$s2	nop	\$t0 ya disponible
4	bne \$s1, \$zero, Loop	sw \$t0, 4(\$s1)	store en delay slot

$IPC = 5 \text{ instrucciones útiles} / 4 \text{ ciclos} = 1,25$ (máximo teórico = 2)

Observaciones:

- El ciclo 1 tiene `nop` en el slot ALU porque la primera instrucción útil disponible (`addi`) no puede ir antes de la carga (la carga necesita el valor de `$s1` sin modificar).
- El `addi` se adelanta al ciclo 2 para cubrir la latencia de carga (*load-use hazard*): cuando `addu` necesita `$t0` en el ciclo 3, la carga ya completó en el ciclo 1 y el forwarding puede actuar.
- La instrucción `sw` se mueve al delay slot del salto (ciclo 4, ranura Load/Store), aprovechando la ranura de retardo y ejecutando una instrucción útil en lugar de una burbuja.

3.3.5 Ejemplo con desenrollado (loop unrolling) para emisión dual

El **desenrollado de bucles** (loop unrolling) para emisión estática multiplica el cuerpo del bucle por un factor k , reduciendo el overhead de control del bucle y exponiendo más instrucciones independientes al compilador. La clave del desenrollado para emisión múltiple es que las k copias del cuerpo del bucle usan registros *distintos* (renombrado de registros manual por el compilador) para evitar generar dependencias WAR y WAW entre iteraciones.

Ejemplo Bucle desenrollado $\times 4$ para MIPS dual estático

Con factor de desenrollado 4, procesando 4 elementos del array por iteración:

Cuadro 3.3: Bucle desenrollado $\times 4$ ($IPC = 14/8 = 1,75$)

Ciclo	Slot ALU/Salto	Slot Load/Store
1	addi \$s1, \$s1, -16	lw \$t0, 0(\$s1)
2	nop	lw \$t1, 12(\$s1)
3	addu \$t0, \$t0, \$s2	lw \$t2, 8(\$s1)
4	addu \$t1, \$t1, \$s2	lw \$t3, 4(\$s1)
5	addu \$t2, \$t2, \$s2	sw \$t0, 16(\$s1)
6	addu \$t3, \$t3, \$s2	sw \$t1, 12(\$s1)
7	nop	sw \$t2, 8(\$s1)
8	bne \$s1, \$zero, Loop	sw \$t3, 4(\$s1)

14 instrucciones útiles en 8 ciclos: $IPC = 1,75$

El IPC mejora de 1,25 (sin desenrollado) a 1,75 (con desenrollado $\times 4$). El acercamiento al máximo teórico de 2 tiene un coste: se necesitan 4 registros temporales (\$t0–\$t3) en lugar de 1, y el código binario es 4 veces más largo.

3.4 Procesadores VLIW (*Very Long Instruction Word*)

Definición VLIW

Una arquitectura **VLIW** (*Very Long Instruction Word*) extiende el concepto de emisión dual estática a cualquier número de operaciones paralelas. Una instrucción VLIW es en realidad un *paquete* de múltiples operaciones independientes, cada una destinada a una unidad funcional distinta, empaquetadas en una sola instrucción de longitud fija (que puede ser de 128, 256 o más bits).

Un procesador VLIW típico podría incluir en una sola instrucción:

- Una operación aritmética entera o un salto

- Dos operaciones independientes en punto flotante
- Dos accesos independientes a memoria (carga o almacenamiento)

Ventajas de VLIW:

- El hardware del procesador es significativamente más simple: no hay unidad de detección de riesgos en tiempo de ejecución, no hay lógica de forwarding compleja (el compilador garantiza que no habrá riesgos dentro del paquete). Esto reduce el área del chip y el consumo de potencia.
- La verificación formal de la corrección es más sencilla (el compilador garantiza statically la ausencia de conflictos).

Desventajas de VLIW:

- El compilador debe encontrar suficiente paralelismo para llenar todas las ranuras en todas las instrucciones. Si no hay instrucciones independientes disponibles, las ranuras vacías consumen espacio de código inútilmente (código con muchos nops).
- El **tamaño del código binario** es muy grande, lo que presiona la caché de instrucciones y la memoria.
- El código compilado para una implementación VLIW *no es compatible* con otra implementación del mismo ISA con diferente número de unidades funcionales: si se añade o elimina una unidad, hay que recompilar.
- Los fallos de caché y otras latencias variables en tiempo de ejecución no pueden gestionarse correctamente en compilación, generando NOP defensivos excesivos.

3.5 Emisión múltiple dinámica: procesadores superescalares

Definición Procesador superescalar

Un **procesador superescalar** es aquel capaz de emitir y ejecutar más de una instrucción por ciclo de reloj mediante decisiones tomadas en tiempo de ejecución por el hardware. La CPU examina el flujo de instrucciones, detecta las independencias entre ellas y decide dinámicamente cuántas y cuáles emitir en cada ciclo (de 0 hasta el máximo configurado).

La característica más poderosa de la emisión dinámica es que permite la **ejecución fuera de orden** (*out-of-order execution*): las instrucciones pueden ejecutarse en un orden diferente al que aparecen en el programa, siempre que las dependencias de datos se respeten. Esto permite que instrucciones que están bloqueadas esperando un dato (por un fallo de caché, por ejemplo) no bloqueen a instrucciones posteriores que son independientes.

Ejemplo Ejecución fuera de orden con emisión dinámica

```
1 lw    $t0, 20($s2)    # I1: carga (puede tardar muchos ciclos  
    si falla cache)  
2 addu  $t1, $t0, $t2    # I2: depende de I1 -> debe esperar  
3 sub   $s4, $s4, $t3    # I3: INDEPENDIENTE de I1 y I2  
4 slti  $t5, $s4, 20     # I4: depende de I3 pero no de I1/I2
```

Listing 3.5: El hardware puede reordenar para evitar paradas

Con ejecución fuera de orden, el hardware puede ejecutar I3 e I4 mientras I1 está esperando que su dato llegue de memoria, en lugar de dejar el pipeline inactivo. I2 solo se ejecutará cuando I1 complete.

3.5.1 ¿Por qué no basta con que el compilador planifique?

Podría parecer que si el compilador reordena el código, el hardware no necesita hacerlo. Sin embargo, hay tres razones fundamentales por las que la planificación estática del compilador es insuficiente:

1. **No todas las paradas son predecibles:** los fallos de caché (cuando un dato no está en la caché L1 ni L2 y hay que ir a DRAM) introducen latencias de cientos de ciclos que el compilador no puede predecir porque dependen del patrón de acceso a datos en tiempo de ejecución.
2. **Las instrucciones de salto impiden la planificación estática más allá del bloque básico:** el compilador solo puede reordenar instrucciones dentro de un *bloque básico* (secuencia sin saltos). La longitud media de un bloque básico es de solo 3 a 6 instrucciones, muy insuficiente para explotar el ILP de procesadores modernos con emisión de 4–8 instrucciones por ciclo.
3. **Portabilidad:** diferentes implementaciones del mismo ISA pueden tener latencias distintas. Un código compilado para un Pentium 4 y optimizado para sus latencias específicas puede ser subóptimo en un Core i7.

3.6 Planificación dinámica: técnica del marcador

La **técnica del marcador** (*scoreboarding*) fue introducida en el CDC 6600 (1964) y constituye el primer mecanismo hardware de planificación dinámica. Su objetivo es permitir que las instrucciones se ejecuten fuera de orden *reteniendo* aquellas que tienen conflictos hasta que los conflictos se resuelvan, sin bloquear todo el pipeline.

El marcador centraliza el control: mantiene tablas de estado que registran qué unidades funcionales están ocupadas, qué instrucciones están esperando operandos y qué registros están siendo escritos. Cada instrucción pasa por los siguientes estados:

1. **Emisión (*issue*):** si la unidad funcional requerida está libre y no hay riesgo WAW (ninguna otra instrucción en ejecución escribe en el mismo registro destino), la instrucción se emite a la unidad funcional. En caso contrario, la instrucción *espera* hasta que las condiciones se cumplan. Esto se hace en orden: no se puede emitir la instrucción *j* antes de la instrucción *i* si *i* precede a *j* en el programa.

2. **Lectura de operandos:** en cuanto los registros fuente están disponibles (es decir, ninguna instrucción pendiente va a escribir en ellos), la instrucción lee sus operandos y comienza la ejecución. Este paso se puede hacer fuera de orden: si los operandos de j están listos antes que los de i , j puede comenzar a ejecutarse primero.
3. **Ejecución:** la unidad funcional ejecuta la operación durante los ciclos necesarios.
4. **Escritura del resultado:** cuando la instrucción termina y el marcador confirma que no hay riesgos WAR (ninguna instrucción que debería haber leído el registro destino aún no lo ha hecho), se escribe el resultado en el banco de registros.

¡Atención! Limitación del marcador: no elimina WAW ni WAR

El marcador *detecta* los riesgos WAW y WAR e inserta paradas para evitarlos, pero no los elimina. Esto limita el ILP explotable. La técnica de Tomasulo (sección siguiente) va más allá: *elimina* estos riesgos mediante renombrado dinámico de registros.

3.7 Planificación dinámica: algoritmo de Tomasulo

El **algoritmo de Tomasulo** fue diseñado por Robert Tomasulo para la unidad de coma flotante del IBM System/360 Model 91 (1967). Es el fundamento conceptual de prácticamente todos los procesadores superescalares modernos (Intel, AMD, ARM). Su contribución esencial es el **renombrado dinámico de registros mediante estaciones de reserva**, que elimina los riesgos WAR y WAW y permite una ejecución fuera de orden mucho más agresiva.

3.7.1 Estructura hardware del sistema Tomasulo

[INSERTAR FIGURA]

Diagrama de bloques del sistema Tomasulo. Se muestran: (1) la cola de instrucciones (FIFO), (2) las estaciones de reserva (RS) para cada unidad funcional (ALU, FP suma, FP mult, load/store), (3) las unidades funcionales, (4) el Bus de Datos Común (CDB), (5) el banco de registros con etiquetas de estación de reserva, y (6) las conexiones del CDB con las RS y el banco de registros. Las flechas muestran el flujo de instrucciones y datos. Fuente: diapositivas Tema 3.

Figura 3.3: Arquitectura del sistema Tomasulo. Las estaciones de reserva actúan como buffer y como tabla de renombrado. El CDB distribuye los resultados a todas las RS que los esperan simultáneamente.

Los componentes clave del sistema Tomasulo son:

Estaciones de reserva (RS)

Cada unidad funcional (suma FP, multiplicación FP, división FP, load/store) tiene un conjunto de **estaciones de reserva** asociadas. Una estación de reserva almacena una instrucción pendiente de ejecutarse junto con toda la información necesaria para ejecutarla cuando los operandos estén disponibles. Los campos de cada entrada de una RS son:

Cuadro 3.4: Campos de una entrada en la Estación de Reserva (RS)

Campo	Tipo	Descripción
B	bit	1 si la entrada está ocupada, 0 si está libre
INST	código	Operación a realizar (add.d, mul.d, lw, etc.)
OP1	valor o etiqueta	Primer operando: valor si está listo, o nombre de la RS que lo producirá
OP2	valor o etiqueta	Segundo operando: igual que OP1
V1	bit	1 si OP1 contiene un valor válido; 0 si es una etiqueta pendiente
V2	bit	1 si OP2 contiene un valor válido; 0 si es una etiqueta pendiente
R	bit	1 si la instrucción está lista para ejecutarse (V1=1 y V2=1)

La idea del renombrado dinámico reside en el uso de las **etiquetas de estación de reserva**: en lugar de esperar a que el resultado se escriba en el banco de registros y luego leerlo, la instrucción que espera ese resultado se anota directamente con el identificador de la RS que lo producirá. Cuando la unidad funcional completa la operación y emite el resultado por el CDB, todas las RS que tienen esa etiqueta como operando reciben el valor y actualizan su campo correspondiente, marcándolo como válido.

Bus de Datos Común (CDB)

El **CDB** (*Common Data Bus*) es el mecanismo por el que los resultados de las unidades funcionales se distribuyen simultáneamente a todas las RS que los están esperando y al banco de registros. Cuando una unidad funcional completa una operación, emite en el CDB tanto el resultado como la etiqueta de la RS que lo produjo. Todas las RS que tienen esa etiqueta como operando pendiente lo capturan inmediatamente.

3.7.2 Algoritmo de Tomasulo: tres fases

Fase 1: Emisión (*Issue*)

Se lee la siguiente instrucción de la cola FIFO de instrucciones (las instrucciones se leen en orden del programa). Si hay una RS libre para la unidad funcional requerida, la instrucción se emite a esa RS:

- Si el operando fuente está disponible en el banco de registros (porque ninguna instrucción pendiente lo va a escribir), su valor se copia directamente en el campo OP de la RS y se marca V=1.
- Si el operando no está disponible (porque otra instrucción en una RS lo va a producir), se copia en el campo OP la *etiqueta* de esa RS y se marca V=0. La instrucción quedará en la RS esperando hasta que el CDB traiga el valor.

Este mecanismo implementa el renombrado dinámico: en lugar de escribir “espera el valor de f0”, la RS guarda “espera el valor que producirá la RS número 3”. Si más adelante se emite otra instrucción que también escribe en f0 (riesgo WAW), el banco de registros apuntará a la nueva RS, no a la antigua. El resultado correcto llegará al destino correcto.

¡Atención! La emisión es siempre en orden

Aunque la ejecución y la escritura de resultados pueden ocurrir fuera de orden, la **emisión** siempre se realiza en el orden del programa. Esto garantiza que las etiquetas de dependencia se asignen correctamente (la última escritura a un registro es la que queda registrada como etiqueta activa).

Fase 2: Ejecución (*Execute*)

La instrucción permanece en la RS hasta que $V1=1$ y $V2=1$ (ambos operandos válidos). En ese momento está lista ($R=1$) y puede enviarse a la unidad funcional cuando esta quede libre. La ejecución puede durar múltiples ciclos para operaciones FP. Las reglas adicionales son:

- Los accesos a memoria (cargas y almacenamientos) se mantienen en el *orden del programa* con respecto a otras referencias a memoria. La dirección efectiva se calcula en cuanto el registro base está disponible.
- No se inicia la ejecución de ninguna instrucción hasta que todas las instrucciones de salto que la preceden en el programa han sido resueltas (para garantizar la corrección con especulación, lo cual se gestiona con los ROBs que veremos a continuación).

Fase 3: Escritura del resultado (*Write Result*)

Cuando la unidad funcional completa la ejecución, el resultado se emite en el CDB junto con la etiqueta de la RS. Simultáneamente:

- Todas las RS que tienen esa etiqueta como operando pendiente capturan el valor y marcan $V=1$ (con lo que pueden pasar a estado “listo”).
- El banco de registros actualiza el valor del registro destino *solo si* ese registro sigue apuntando a esta RS (si una instrucción posterior ha reasignado la etiqueta a otra RS por una escritura posterior al mismo registro, el banco de registros no se actualiza: el valor antiguo no es necesario).
- La RS queda libre ($B=0$) para ser usada por nuevas instrucciones.

3.7.3 Ejemplo de renombrado de registros con Tomasulo

Ejemplo Renombrado dinámico para eliminar WAR y WAW

```
1 fdiv.d f0, f2, f4      # escribe f0 (lento: division FP)
2 fadd.d f6, f0, f8      # RAW en f0; escribe f6
3 fsd      f6, 0(x1)     # RAW en f6
4 fsub.d f8, f10, f14    # escribe f8: WAR con fadd.d (que lee
                        # f8)
5 fmul.d f6, f10, f8     # WAW en f6 (fadd.d tb escribe f6);
                        # RAW en f8
```

Listing 3.6: Código con dependencias WAR y WAW en f0, f6 y f8

Tras el renombrado de registros por Tomasulo (asignando RS1 a fdiv.d, RS2 a fadd.d, RS3 a fsd, RS4 a fsub.d, RS5 a fmul.d):

```
1 fdiv.d RS1, f2, f4     # RS1 producira el nuevo f0
2 fadd.d RS2, RS1, f8    # espera RS1; f8 es el ACTUAL f8 (de
                        # antes de fsub.d)
3 fsd      RS2, 0(x1)    # espera RS2; sin conflicto
4 fsub.d RS4, f10, f14   # RS4 producira el nuevo f8; no afecta
                        # a RS2
5 fmul.d RS5, f10, RS4   # espera RS4; RS5 producira el nuevo
                        # f6 (no afecta a RS2)
```

Listing 3.7: Código equivalente tras renombrado (notación conceptual)

Las dependencias WAR (f8 entre fadd.d y fsub.d; f6 entre fadd.d y fmul.d) y WAW (f6 entre fadd.d y fmul.d) han desaparecido. Solo quedan las dependencias RAW (flujo real de datos), que no se pueden eliminar y que el sistema Tomasulo gestiona esperando a que las etiquetas se resuelvan en el CDB.

3.8 Especulación basada en hardware

La planificación dinámica por sí sola tiene un problema fundamental: si las instrucciones se ejecutan fuera de orden y se producen excepciones, puede ser imposible determinar qué instrucciones se habían completado y cuáles no. Además, si el procesador quiere explotar especulación de salto (ejecutar instrucciones de la rama predicha antes de saber si la predicción es correcta), necesita un mecanismo para deshacer el trabajo especulativo si la predicción falla.

La solución son los **Buffers de Reordenamiento** (ROB, *Reorder Buffers*).

3.8.1 Buffers de reordenamiento (ROB)

Definición Buffer de reordenamiento (ROB)

Un **ROB** es un buffer circular que almacena las instrucciones emitidas en el orden del programa, junto con sus resultados calculados (pero aún no “comprometidos” al estado arquitectónico). El ROB garantiza que los resultados se escriben en el banco de registros *en orden*, aunque las instrucciones se hayan ejecutado fuera de orden.

La diferencia fundamental entre Tomasulo sin ROB y Tomasulo con ROB es que sin ROB los resultados se escriben directamente en el banco de registros al terminar la ejecución (sin garantizar el orden), mientras que con ROB los resultados se depositan primero en el ROB y solo se “comprometen” al banco de registros (*commit*) cuando lleguen al frente del ROB (es decir, cuando todas las instrucciones anteriores ya se hayan comprometido).

3.8.2 Ciclo de vida de una instrucción con ROB

Una instrucción en un procesador superescalar moderno con Tomasulo+ROB pasa por los siguientes estados bien diferenciados:

1. **En ejecución:** la instrucción ha sido emitida a una estación de reserva y está esperando operandos o siendo ejecutada en la unidad funcional.
2. **Finalizada (*completed*):** la unidad funcional ha terminado el cálculo. El resultado se escribe en el campo del ROB correspondiente a esta instrucción (y en las RS que lo esperaban, vía CDB). El banco de registros arquitectónico *aún no* se actualiza.
3. **Comprometida (*committed*):** la instrucción llega al frente del ROB (todas las instrucciones anteriores ya se comprometieron) y su resultado se escribe definitivamente en el banco de registros (si el destino es un registro) o en la memoria caché de datos (si es un almacenamiento). La instrucción queda retirada del ROB.
4. **Retirada (*retired*):** coincide con “comprometida” si el destino es un registro. Si el destino es memoria, la retirada ocurre cuando la escritura llega a la caché de datos.

3.8.3 Especulación con ROB: saltos y cargas

Especulación de salto

Con ROB, el procesador puede emitir instrucciones después de un salto condicional antes de saber si el salto se tomará o no (especulación). Los resultados de esas instrucciones especulativas se depositan en el ROB pero no se comprometen al banco de registros. Si la predicción es correcta, cuando el salto se resuelve las instrucciones especulativas ya estaban ejecutándose: ganancia de ciclos. Si la predicción es incorrecta, se vacía el ROB desde la instrucción después del salto hasta el final: todas

[INSERTAR FIGURA]

Diagrama de bloques del sistema Tomasulo con ROB. Se muestran: (1) cola de instrucciones, (2) lógica de emisión y asignación de entradas ROB, (3) estaciones de reserva conectadas a unidades funcionales, (4) el CDB que escribe resultados tanto en RS como en el ROB, (5) el ROB como buffer circular con punteros head/tail, y (6) la lógica de commit que escribe del ROB al banco de registros en orden. Fuente: diapositivas Tema 3.

Figura 3.4: Sistema Tomasulo con Buffers de Reordenamiento (ROB). Las instrucciones se ejecutan fuera de orden pero se comprometen en orden. La especulación es posible porque los resultados especulativos solo residen en el ROB hasta que se confirman.

las instrucciones especulativas se descartan y el banco de registros permanece inalterado (nunca fueron comprometidas).

Especulación de carga (*load speculation*)

La especulación de carga busca evitar los retardos debidos a fallos de caché:

1. Se predice la dirección efectiva del `lw` (o se usa la que se calcula especulativamente).
2. Se carga el valor de esa dirección.
3. El resultado especulativo se deposita en el ROB.
4. No se asigna al banco de registros hasta que se confirma que la dirección era correcta y no hubo fallo de caché que requiriera corrección.

3.9 Arquitectura de alto nivel de un núcleo superescalar moderno

Un núcleo superescalar moderno (como los usados en el Intel Core i7 o el ARM Cortex-A8) integra todos los mecanismos estudiados en los Temas 2 y 3:

El flujo de instrucciones en un procesador superescalar con Tomasulo+ROB es:

1. **Búsqueda múltiple (IF):** se captan s instrucciones por ciclo de la caché de instrucciones, usando el predictor de saltos para anticipar qué instrucciones vienen a continuación.
2. **Decodificación múltiple (ID):** se decodifican s instrucciones simultáneamente.
3. **Distribución y renombrado:** cada instrucción decodificada se asigna a una estación de reserva (RS) libre y se le asigna una entrada en el ROB. Se realiza el renombrado de registros.

[INSERTAR FIGURA]

Diagrama de bloques de un núcleo superescalar moderno con: (1) caché L1 de instrucciones, (2) unidad de búsqueda de múltiples instr., (3) predictor de saltos dinámico, (4) decodificación de múltiples instrucciones, (5) renombrado de registros y asignación de ROB/RS, (6) unidades de ejecución paralelas (ALU×2, FP, Load/Store), (7) CDB, (8) ROB circular, (9) lógica de commit y banco de registros arquitectónico, (10) caché L1 de datos. Fuente: diapositivas Tema 3.

Figura 3.5: Arquitectura de un núcleo superescalar moderno. El camino de datos se puede dividir conceptualmente en el *front-end* (búsqueda, decodificación, renombrado) y el *back-end* (emisión, ejecución, escritura, commit).

4. **Ejecución fuera de orden:** las instrucciones se envían a las unidades funcionales en cuanto sus operandos están disponibles ($R=1$), sin importar el orden del programa.
5. **Escritura del resultado:** los resultados se emiten por el CDB al ROB y a las RS que los esperan.
6. **Commit en orden:** las instrucciones se comprometen al banco de registros en el orden del programa, cuando llegan al frente del ROB y se confirma que su resultado es correcto (no especulativo).

3.9.1 Comparativa: ARM Cortex-A8 vs Intel Core i7

Las diapositivas presentan una comparativa directamente relevante para el examen:

Nota de examen Importante: no confundir profundidad de pipeline con rendimiento

Ambos procesadores tienen **14 etapas de pipeline**, pero sus rendimientos son muy diferentes. El Core i7 es mucho más rápido porque: (a) tiene 4 núcleos, (b) emite hasta 4 instrucciones por ciclo (frente a 2 del A8), (c) usa ejecución fuera de orden con especulación, y (d) tiene mayor frecuencia. El número de etapas del pipeline afecta principalmente a la latencia de cada instrucción y a la penalización por salto, no al throughput.

3.10 Eficiencia energética en la emisión múltiple

La complejidad creciente de los mecanismos de emisión dinámica, ejecución fuera de orden y especulación tiene un coste energético considerable. La tabla histórica de las diapositivas ilustra esta evolución con precisión quirúrgica:

Observaciones clave de la tabla:

Cuadro 3.5: Comparativa ARM Cortex-A8 (PMD) vs Intel Core i7 (servidor)

Característica	ARM Cortex-A8	Intel Core i7 920
Mercado	PMD (móvil, embebido)	Servidor, cloud
TDP	2 W	130 W
Frecuencia	1 GHz	2,66 GHz
Núcleos	1	4
Punto flotante	No	Sí
Emisión múltiple	Dinámica	Dinámica
Instrucciones máx./ciclo	2	4
Etapas de pipeline	14	14
Planificación	Estática en orden	Dinámica fuera de orden
Predicción de saltos	2 niveles	2 niveles
Caché L1/núcleo	32 KB I + 32 KB D	32 KB I + 32 KB D
Caché L2/núcleo	128–1024 KB	256 KB
Caché L3 compartida	–	2–8 MB

Cuadro 3.6: Evolución de procesadores Intel: complejidad vs. potencia

Procesador	Año	Frec.	Etapas	IPC	OoO?	TDP
i486	1989	25 MHz	5	1	No	5 W
Pentium	1993	66 MHz	5	2	No	10 W
Pentium Pro	1997	200 MHz	10	3	Sí	29 W
P4 Willamette	2001	2000 MHz	22	3	Sí	75 W
P4 Prescott	2004	3600 MHz	31	3	Sí	103 W
Core 2006	2006	2930 MHz	14	4	Sí	75 W
UltraSparc III	2003	1950 MHz	14	4	No	90 W
UltraSparc T1	2005	1200 MHz	6	1	No	70 W

1. El Pentium 4 Prescott (2004) consume 103 W con solo $IPC = 3$ y 3600 MHz. Esto ilustra el muro de potencia que forzó la transición al multinúcleo (Tema 1): seguir aumentando la frecuencia y la profundidad del pipeline se volvió energéticamente insostenible.
2. El Core (2006) consume solo 75 W a pesar de tener el mismo IPC que el Prescott y una frecuencia similar. La clave fue reducir la profundidad del pipeline de 31 a 14 etapas, reduciendo drásticamente la frecuencia de reloj necesaria y el consumo dinámico.
3. El UltraSparc T1 adopta la estrategia opuesta a la del Pentium 4: pipeline muy corto (6 etapas), $IPC = 1$ por núcleo, pero 8 núcleos. Para aplicaciones que pueden paralelizarse, el throughput es comparable con un consumo mucho menor por núcleo.

Concepto clave El equilibrio entre ILP y energía

Los mecanismos de emisión dinámica, ejecución fuera de orden y especulación requieren energía adicional proporcional a su complejidad. El diseño moderno equilibra el ILP con la eficiencia energética, y cuando no es posible mejorarlo, opta por integrar más núcleos simples (*many-core*) en lugar de un núcleo más complejo. Este es precisamente el principio que estudiaremos en el Tema 4.

3.11 Falacias sobre el ILP: errores conceptuales frecuentes en el examen

Las diapositivas dedican explícitamente una sección a las “falacias”, lo que indica que son confusiones frecuentes que conviene aclarar en detalle.

3.11.1 Falacia 1: un CPI más bajo o una frecuencia más alta siempre implica mayor rendimiento

Esta confusión surge de aplicar la ecuación $T_{CPU} = N \cdot CPI \cdot T_{ciclo}$ de forma parcial. El CPI y la frecuencia no son independientes entre sí ni del número de instrucciones N .

El **Pentium 4** es el ejemplo paradigmático: tiene un CPI más bajo que sus contemporáneos, pero era más lento en benchmarks reales porque su pipeline de 22–31 etapas aumentaba masivamente la penalización por saltos mal predichos (penalización proporcional al número de etapas), y su ISA x86 generaba muchas más instrucciones que arquitecturas RISC para el mismo programa.

El **Itanium** (IA-64, VLIW) tiene CPI similar a sus contemporáneos pero con frecuencia más baja. Sin embargo, al ser VLIW, sus instrucciones son más anchas (hacen más trabajo por instrucción), por lo que N es mucho menor. El rendimiento efectivo no puede deducirse de CPI o frecuencia aisladamente.

¡Atención! Regla de oro: la única métrica fiable es el tiempo de ejecución

Siempre que se compare el rendimiento de dos procesadores, la única métrica correcta es el **tiempo de ejecución del mismo programa** en las mismas condiciones. CPI, IPC, MIPS y frecuencia de reloj son métricas parciales que pueden inducir a error si se analizan por separado.

3.11.2 Falacia 2: hay grandes cantidades de ILP disponibles que se podrían explotar con técnicas mejores

Esta falacia llevaría a creer que podríamos construir procesadores con $IPC = 10, 20$ o más simplemente mejorando los algoritmos de planificación. La realidad es que el ILP disponible en los programas reales es fundamentalmente limitado por las dependencias de datos reales (RAW), que no pueden eliminarse con ninguna técnica de renombrado o reordenamiento.

Las diapositivas muestran una gráfica de ILP potencial en función del tamaño de la ventana de instrucciones analizada. Las conclusiones son:

- Para ventanas pequeñas (4–8 instrucciones, correspondientes a procesadores reales), el ILP explotable es de 2–4 instrucciones por ciclo.
- Incluso con ventanas infinitas (analizando todo el programa), el ILP potencial raramente supera 8–10 instrucciones por ciclo en programas de propósito general.
- Las dependencias reales (RAW), los accesos a memoria con latencias variables y los saltos condicionales son los factores limitantes fundamentales.

3.12 Ejercicios resueltos con metodología de examen

Ejercicio resuelto 1 — Impacto del predictor de saltos sobre el CPI

Enunciado. Mezcla de instrucciones: tipo R 40%, BEQ 25%, J 5%, LW 25%, SW 5%. Precisiones de los predictores: “siempre tomado” 45%, “siempre no tomado” 55%, predictor de 2 bits 85%. Pipeline como el MIPS: dirección de salto calculada en ID, decisión en EX (1 ciclo de penalización por predicción incorrecta). CPI base = 1, sin riesgos de datos, sin ranuras de retardo.

Fórmula general:

$$CPI_{ef} = 1 + f_{BEQ} \times (1 - \text{tasa_acierto}) \times \text{penalización}$$

donde $f_{BEQ} = 0,25$ y $\text{penalización} = 1$ ciclo.

Apartado a) Predictor “siempre tomado” (tasa de acierto = 45%):

$$CPI_a = 1 + 0,25 \times (1 - 0,45) \times 1 = 1 + 0,25 \times 0,55 = 1 + 0,1375 = \mathbf{1,1375}$$

Apartado b) Predictor “siempre no tomado” (tasa de acierto = 55%):

$$CPI_b = 1 + 0,25 \times (1 - 0,55) \times 1 = 1 + 0,25 \times 0,45 = 1 + 0,1125 = \mathbf{1,1125}$$

Apartado c) Predictor de 2 bits (tasa de acierto = 85%):

$$CPI_c = 1 + 0,25 \times (1 - 0,85) \times 1 = 1 + 0,25 \times 0,15 = 1 + 0,0375 = \mathbf{1,0375}$$

Comparación y conclusiones:

Cuadro 3.7: Comparativa de predictores de salto por CPI efectivo

Predictor	Tasa de acierto	ΔCPI	CPI efectivo
Siempre tomado	45%	0,1375	1,1375
Siempre no tomado	55%	0,1125	1,1125
Predictor 2 bits	85%	0,0375	1,0375

El predictor de 2 bits reduce el ΔCPI por saltos mal predichos en un factor de $0,1375/0,0375 \approx 3,7\times$ respecto al predictor “siempre tomado”, y de $0,1125/0,0375 = 3\times$ respecto al “siempre no tomado”. Esto justifica el hardware adicional del predictor dinámico.

Ejercicio resuelto 2 — Predictor de 2 bits: simulación ciclo a ciclo

Enunciado. Secuencia repetitiva de resultados de salto: T, NT, T, T, NT (donde T = tomado, NT = no tomado). Se usa la siguiente máquina de estados del predictor de 2 bits:

Estado 0 (Fuertemente NT) $\leftrightarrow$ Estado 1 (Débilmente NT) $\leftrightarrow$ Estado 2 (Débilmente T) $\leftrightarrow$ Estado 3 (Fuertemente T)

Estado inicial: 0 (“Predict not taken”).

Apartado a) Precisión de predictores fijos:

- Siempre tomado: acierta en T, T, T (3 de 5) = **60%**
- Siempre no tomado: acierta en NT, NT (2 de 5) = **40%**

Apartado b) Predictor de 2 bits, 4 primeros saltos (estado inicial = 0):

Cuadro 3.8: Predictor 2 bits: primeras 4 ocurrencias, estado inicial = 0

Salto	Estado	Predicción	Resultado	Correcto?	Nuevo estado
1	0	NT	T	Incorrecto	1
2	1	NT	NT	Correcto	0
3	0	NT	T	Incorrecto	1
4	1	NT	T	Incorrecto	2

$$\text{Precisión} = 1/4 = \mathbf{25\%}$$

Apartado c) Estado estacionario:

Para determinar la precisión en estado estacionario hay que simular hasta que el estado al inicio de una ocurrencia del patrón se repita:

Cuadro 3.9: Predictor 2 bits: búsqueda del estado estacionario

Ocurrencia	Estados al inicio de cada salto					Resultados (C=correcto, I=incorrecto)				
	T	NT	T	T	NT	T	NT	T	T	NT
1 ^a	0	1	0	1	2	I	C	I	I	I
2 ^a	1	2	1	2	3	I	C	I	C	I
3 ^a	2	3	2	3	3	I	C	I	C	I
4 ^a	2	3	2	3	3	I	C	I	C	I

Estado estacionario a partir de la 3^a ocurrencia (estados iniciales se repiten). En estado estacionario: C, I, C, C, I → **3 de 5 = 60%** de aciertos.

Apartado d) Predictor perfecto para este patrón:

Un predictor perfecto para el patrón T-NT-T-T-NT (que se repite indefinidamente) sería un **registro de desplazamiento circular de 5 bits** inicializado con el patrón: [1, 0, 1, 1, 0] (1 = T, 0 = NT). La predicción siempre es el bit más a la izquierda, y tras cada salto el registro se desplaza. Como el patrón se repite, el registro vuelve siempre al estado inicial: la precisión es del 100%.

3.13 Resumen y conclusiones del Tema 3

Los conceptos fundamentales que el alumno debe dominar para el examen son:

1. **ILP e IPC:** la emisión múltiple permite $CPI < 1$, y por ello se usa el IPC como métrica. La fórmula de CPI efectivo acumula las contribuciones de los tres tipos de parada.
2. **Dependencias de nombre (WAR y WAW):** son artificiales y se eliminan con renombrado de registros (manual en el compilador para emisión estática, dinámico mediante RS en Tomasulo).
3. **Emisión estática (MIPS dual, VLIW):** el compilador construye paquetes de instrucciones independientes. El hardware es simple pero el compilador necesita conocer el hardware. Útil para aplicaciones científicas.
4. **Emisión dinámica (superescalar):** el hardware detecta y gestiona los riesgos en tiempo de ejecución. Permite ejecución fuera de orden. Mayor complejidad y consumo, pero mayor rendimiento en aplicaciones generales.
5. **Marcador (scoreboard):** primera técnica de planificación dinámica. Detecta y espera los riesgos WAW y WAR, pero no los elimina.
6. **Tomasulo:** eliminación de WAR y WAW mediante renombrado dinámico con estaciones de reserva (RS). El CDB distribuye resultados. Las tres fases: emisión, ejecución y escritura de resultado.

7. **ROB + Tomasulo:** añade soporte para excepciones precisas y especulación. Las instrucciones se ejecutan fuera de orden pero se *comprometen* en orden. Los estados clave: en ejecución, finalizada, comprometida/retirada.
8. **Eficiencia energética:** más ILP = más potencia. La complejidad del hardware de planificación dinámica y especulación consume energía de forma cuadrática aproximadamente con la anchura de emisión.
9. **Falacias:** CPI y frecuencia no determinan el rendimiento de forma aislada. El ILP explotable en programas reales está fundamentalmente limitado por las dependencias RAW y los accesos a memoria.

Fórmulas y reglas rápidas para el examen — Tema 3

CPI efectivo con todas las penalizaciones:

$$CPI_{ef} = CPI_{ideal} + \Delta CPI_{estructural} + \Delta CPI_{datos} + \Delta CPI_{control}$$

Contribución de saltos mal predichos al CPI:

$$\Delta CPI_{ctrl} = f_{saltos} \times (1 - \text{tasa_acierto}) \times \text{penalización_ciclos}$$

IPC para emisión múltiple:

$$IPC = \frac{1}{CPI} \leq \text{anchura_emisión}$$

Speedup del desenrollado con factor k :

$$S = \frac{\text{ciclos_original} \times k}{\text{ciclos_desenrollado}}$$

Condiciones de forwarding en las RS de Tomasulo:

- Emisión: copiar operando si está en banco de reg. o en ROB; si no, copiar etiqueta.
- Ejecución: cuando $V1=V2=1$ ($R=1$) y unidad libre $\rightarrow$ ejecutar.
- Escritura: emitir por CDB con etiqueta; actualizar RS y ROB que esperan.
- Commit: escribir en banco de reg. en orden desde el frente del ROB.

Tipos de dependencia y tratamiento:

Tipo	Nombre	¿Ocurre en MIPS?	Solución
RAW	Verdadera	Sí	Forwarding / espera
WAR	Antidependencia	No (enteros) / Sí (OoO)	Renombrado
WAW	Salida	No (enteros) / Sí (FP)	Renombrado